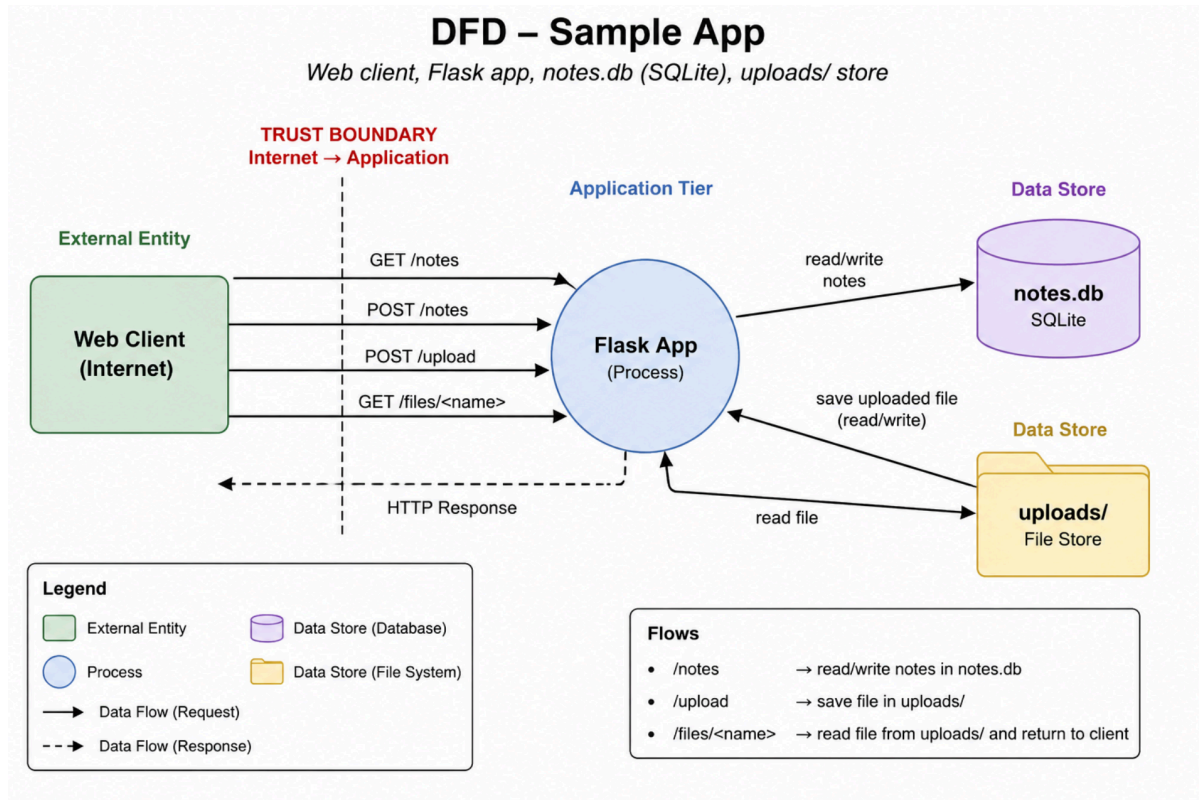


Threat Model — Flask Sample Application (Week 1)

1. Data-flow diagram



(Trust boundary crossed: Internet → Application process boundary)

2. Elements & trust boundaries

Element	Type (process/store/entity/flow)	Trust boundary crossed?
Web client	external entity	yes (Internet → Flask app boundary)
Flask app (app.py)	process	yes (receives untrusted external HTTP inputs)
SQLite DB (notes.db)	data store	no (internal process storage)
uploads/ store	data store	yes (accessible via HTTP serving & path traversal)
/notes flow	data flow	yes (untrusted JSON input → DB insert & full read)
/upload flow	data flow	yes (untrusted file payload → raw disk write)
/files/<name> flow	data flow	yes (untrusted URL param → file read)

3. STRIDE analysis

Element	Spoofing (S)	Tampering (T)	Repudiation (R)	Information Disclosure (I)
/notes	Client supplies arbitrary owner string	Unauthenticated clients can insert arbitrary or corrupted note bodies into	No request logging or	GET / POST returns all

Element	Spoofing (S)	Tampering (T)	Repudiation (R)	Information Disclosure (I)
	<code>(request.json.get("owner"))</code> without authentication; anyone can impersonate any user.	<code>notes.db</code> .	audit trail; impossible to prove who created or modified a note.	records from <code>notes.db</code> (<code>SELECT id, owner, body FROM notes</code>) to any unauthenticated caller.
/upload	Upload endpoint is unauthenticated; anonymous clients can upload files claiming to be system files or other users.	<code>f.save(os.path.join(UPLOAD_DIR, f.filename))</code> uses raw filename. Path traversal (<code>../../app.py</code>) allows arbitrary file write/overwrite.	No upload logging (no client IP, timestamp, or file hash logged); actions cannot be traced back to caller.	Echoes back internal save paths in response <code>{"saved": f.filename}</code> ; error tracebacks expose filesystem structure.
/files/	File retrieval is unauthenticated; anyone can request any filename without identity verification.	Low for direct path traversal on read (defended by Werkzeug <code>send_from_directory</code>), but reads tampered files uploaded via <code>/upload</code> .	No read/download access logging; cannot trace who retrieved sensitive uploaded files.	Unrestricted public access to <code>uploads/</code> directory; any user who knows/guesses a filename can download sensitive files.
SQLite DB (notes.db)	No DB-level authentication; Flask process opens file directly.	Database rows modified by unvalidated web input.	SQLite engine does not log row insertions or access attempts.	Database file stored unencrypted on disk; readable if server filesystem is compromised.
uploads/store	No file ownership or metadata enforced on disk.	Files can be overwritten directly if filenames collide or path traversal occurs.	No filesystem file modification auditing.	Web-accessible directory permits directory listing or direct download if names are known.

4. Top 5 risks (likelihood × impact) + mitigation

1. Arbitrary File Write / Path Traversal via /upload (Tampering / Elevation of Privilege)

- *Risk*: High Likelihood × Critical Impact.
- *Mitigation*: Sanitize input filenames using `werkzeug.utils.secure_filename()`, validate file extensions against an allow-list, and store uploaded files outside the web root.

2. Broken Access Control & Identity Spoofing on /notes (Spoofing / Information Disclosure)

- *Risk*: High Likelihood × High Impact.
- *Mitigation*: Implement session-based authentication (e.g. JWT / Cookies) and enforce authorization checks so users can only view and create their own notes.

3. Unrestricted File Upload / Disk Exhaustion DoS on /upload (Denial of Service)

- *Risk*: High Likelihood × High Impact.
- *Mitigation*: Enforce `app.config['MAX_CONTENT_LENGTH']` limit (e.g., 2MB) and implement IP-based rate limiting.

4. Information Disclosure of All Notes on /notes (Information Disclosure)

- *Risk*: High Likelihood × High Impact.
- *Mitigation*: Scope SELECT queries to the authenticated user ID (`WHERE owner = ?`) instead of returning `SELECT *`.

5. Absence of Audit Logging (Repudiation)

- *Risk*: Medium Likelihood × Medium Impact.
- *Mitigation*: Implement centralized application logging capturing timestamp, client IP, user ID, endpoint, and action for all state-changing requests (POST).